

Regime-Aware Policy Transfer for Computationally Efficient Adaptation under Network Data Drift

Anonymous Submission
Paper ID: *under double-blind review*

Abstract—Network telemetry is non-stationary, and predictive models trained on an earlier period degrade when operating conditions change. Conventional adaptation responds by retraining, which is effective but repeatedly pays the full cost of rebuilding a model. We examine whether adaptation policies learned for previously observed operating regimes can be reused instead of rebuilt. We present Regime-Aware Policy Transfer (RAPT), which associates a learned ensemble policy with each regime, retrieves that policy when a matching regime recurs, and refreshes it when monitoring indicates the stored policy no longer fits. RAPT is evaluated against frozen, event-driven and full-retraining baselines on four network streams under a strictly prequential protocol. Across the four streams RAPT reduced adaptation CPU relative to full retraining by 48–92%, but its predictive performance was not uniformly better: it recorded lower macro-F1 on all four streams, and on one stream blind reuse was actively harmful, a failure that a parity-refit mechanism repairs. A cost-aware configuration matched full retraining within measurement noise ($\Delta_{\text{macro-F1}} = +0.0022$, $p = 0.31$) at roughly 40% lower adaptation cost. Two mechanisms were rejected on measured evidence: a relative-degradation trigger that cannot fire on persistently poor policies, and an incremental learner that was dominated on both accuracy and cost.

Index Terms—concept drift, network telemetry, online learning, adaptive ensembles, regime-aware learning, computationally efficient adaptation

I. INTRODUCTION

Operational network data rarely holds still. Load, radio conditions, subscriber mobility and configuration changes all shift the distribution of the measurements that monitoring systems observe, and a classifier fitted to one period of a 5G testbed or an ISP netflow archive will not stay accurate indefinitely. The standard remedy is to retrain when performance falls. That works, but it is paid for repeatedly, and on high-rate telemetry the cumulative rebuild cost can exceed the cost of serving predictions. Keeping that cost within budget is a recurring concern in streaming deployments, and lightweight detection-and-adaptation frameworks have been proposed specifically for constrained devices [16].

The streaming-learning literature has produced effective detectors for *when* to adapt, including ADWIN [7], Page-Hinkley tests [8], error-rate methods such as DDM and EDD [9], [10], and distributional tests based on exponentially weighted statistics or distance measures [12], [14], [15]. What these detectors do not provide is a way to exploit the structure of the change. Network conditions are frequently *recurring* rather than merely *novel*: a weekday congestion profile, a radio configuration, or an operator-specific QoS regime may appear, disappear, and return. Recurring concepts motivated

some of the earliest work in the field [4], [5], yet the common response remains to relearn. When a previously encountered regime returns, the model trained for it is often still the right model, and rebuilding an equivalent one discards information that was already paid for.

This paper studies whether that information can be retained and reused. RAPT maintains a repository of adaptation policies indexed by operating regime. At a regime boundary it checks whether a policy for the incoming regime is already stored; if so, the stored ensemble is retrieved and used directly at near-zero adaptation cost, and otherwise a policy is trained and added to the repository. Because a retrieved policy may no longer fit if the regime’s label relationship has changed, RAPT also monitors the retrieved policy’s streaming accuracy and can refresh it.

The mechanism is deliberately narrow. RAPT is not a new classifier and does not change how the ensemble is trained; it changes when that training is paid for. The empirical question is whether the substitution is safe, and our answer is conditional. Where regimes recur with stable label semantics, reuse preserves accuracy while removing most of the adaptation cost; where regimes recur but the feature-to-label relationship has shifted, reuse retrieves a stale policy and degrades performance, and the enhancement that repairs it is what makes the approach viable there. We report both outcomes, plus two mechanisms we implemented, measured, and rejected.

The contributions are as follows. First, a regime-conditioned policy-transfer mechanism that stores learned ensemble policies by regime and retrieves them on recurrence, avoiding full retraining when a suitable prior policy exists. Second, a monitoring and refresh strategy for detecting when a retrieved policy has stopped fitting, with a cost-aware refresh configuration that retains the accuracy of a full refit at a fraction of its cost. Third, an empirical evaluation across four network streams under one prequential protocol, covering predictive performance, adaptation cost, policy reuse, and the recurrence structure the mechanism depends on, together with a comparison against four established drift detectors that shows detector-driven adaptation and policy reuse trade accuracy against cost rather than one dominating the other. Fourth, a failure analysis covering a case where blind reuse is harmful, a trigger that cannot fire by construction, and an incremental learner dominated on both axes.

II. RELATED WORK

A. Concept and Data Drift

Distributional change in streaming data is conventionally separated into covariate shift, where $P(X)$ changes while $P(Y | X)$ is stable, and concept drift, where $P(Y | X)$ itself changes [1], [2]. Surveys of the area document a long history of taxonomies and adaptation strategies [3], beginning with the recurring concepts of the STAGGER system [4] and the robustness–flexibility trade-off studied by Widmer [5]. The two call for different responses, since covariate shift can often be absorbed by recalibrating a decision boundary whereas concept drift invalidates the mapping the model learned. Recurring drift, in which an earlier distribution reappears, has received less attention than gradual or abrupt drift despite being common in systems whose load and configuration follow periodic patterns [6].

B. Drift Detection

Detectors provide the trigger for adaptation rather than the adaptation itself. ADWIN maintains an adaptive window over a stream and detects change by comparing sub-window means [7]. The Page-Hinkley test is a sequential cumulative-sum procedure from industrial quality control [8]. DDM and its successor EDD monitor the error rate and its variance, signalling when the error distribution departs from its observed minimum [9], [10], while RDDM reacts to the same statistics with reduced false alarms [11]. EDMA tracks an exponentially weighted moving average of the error against its own dispersion, an idea also exploited by EWMA-based control charts [12]. Label-free detectors instead test the feature stream, using Hoeffding bounds [13], an incremental Kolmogorov–Smirnov statistic [14], or the earth-mover distance [15], and lightweight versions of these tests have been adapted for constrained IoT deployments [16]. These detectors differ in sensitivity and in how often they fire, and that difference translates directly into adaptation cost.

C. Adaptive and Online Learning

Online ensembles adapt by updating constituent models as data arrives. Dynamic weighted majority weights its learners by recent accuracy and adds or removes them as the stream evolves [17], an idea that disagreement-minimizing methods had already applied to drifting concepts [18]. Online bagging and boosting approximate batch resampling under a streaming constraint, and adaptive random forests extend this with drift-aware tree replacement [6]. Rebuilding component weights in response to drift is the basis of the accuracy-updated ensemble [19], and ensemble designs of this kind have been specialised to social big-data and IoT streams [20], [21]. Single-pass incremental learners such as Hoeffding trees bound memory and update cost per example [22], and the River library provides practical implementations of both the detectors and the incremental learners used here [23]. Incremental updating is attractive because its per-example cost is independent of accumulated history, but it is a different mechanism from policy reuse: it changes *how* a model is updated rather than whether

an already-adequate model can be retrieved. Section VI-D reports a direct comparison, in which incremental updating did not improve the accuracy–cost trade-off on the stream we evaluated.

D. Transfer under Recurring Conditions

Transfer learning moves representations or parameters from a source task to a related target task [24], where the object transferred is model structure or feature representation. Retaining rather than discarding previously learned structure has an older lineage in work on the role of forgetting [25] and on the cost of reusing compiled knowledge chunks [26], both of which observe that relearning what has already been learned is wasteful. RAPT transfers something narrower: a previously fitted policy, together with the regime identity under which it was fitted, is stored and later retrieved as a whole. The decision RAPT makes is not how to adapt a model but whether adaptation is needed at all, given that a candidate policy already exists. Where recurrence is absent the mechanism degenerates to ordinary retraining, as we observe on the stream with the fewest regime transitions.

III. PROBLEM FORMULATION

Let a stream be a sequence of observations in chronological order, $x_1, x_2, \dots$, where $x_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \{1, \dots, C\}$ its label, available after the prediction for x_t has been made. The stream is partitioned into consecutive fixed-size windows $W_1, \dots, W_T$, and window W_k carries a regime identifier $r_k \in \mathcal{R}$. A regime is a semantically defined operating condition rather than a cluster label: a network scenario for the 5G testbed stream, a workday/weekend and time-of-day block for the netflow stream, and a mobile network operator for the cross-border trace.

A *policy* π is a fitted predictive model with its decision rule, and the policy repository is a mapping $\mathcal{P} : \mathcal{R} \rightarrow \pi$ populated as regimes are encountered. When the stream moves from r_{k-1} to r_k , the controller chooses an action $a_k \in \{\text{REUSE}, \text{TRAIN}\}$: reuse $\mathcal{P}(r_k)$ if it exists, or train a new policy from the recent buffer. With $M(\pi, W)$ a performance measure (macro-F1 here) and $c(\pi)$ the cost of producing π in adaptation CPU time, the problem is to keep $M(\cdot, W_k)$ close to what a freshly trained policy would achieve while minimizing $\sum_k c(a_k)$.

The difficulty is that reuse is appropriate only while the stored policy still fits the regime. Reuse of $\mathcal{P}(r_k)$ assumes $P(Y | X, r_k)$ is unchanged since the policy was stored. If the regime identifier recurs but the conditional relationship has shifted, that assumption fails and reuse is silently harmful, because the retrieved policy is applied without any signal that it is stale. Distinguishing recurrence of the regime from recurrence of the label relationship is the central difficulty the controller must handle.

IV. PROPOSED METHOD: RAPT

Fig. 1 shows the overall data flow, from window construction through regime matching to policy reuse or fallback.

A. Streaming Architecture

RAPT sits between the stream and the predictive model and makes one decision per regime boundary. Observations arrive in order; each window is predicted with the currently active policy and evaluated once its labels are available. The active policy changes only at a regime transition or when monitoring reports degradation. This ordering, predict then evaluate then adapt, is the prequential discipline described in Section V and is what prevents a window's labels from influencing its own prediction.

B. Regime Representation

A regime is represented by its identifier, derived from the semantic annotation carried by the stream. Using the semantic label rather than a learned clustering keeps the repository keys stable and interpretable, and avoids the failure mode in which a clustering algorithm splits one operating condition into several keys and defeats reuse. The consequence is that the mechanism depends on the stream providing a usable regime signal, an assumption we return to in Section VIII.

C. Policy Repository

The repository stores, for each regime identifier, a checkpoint containing the fitted heterogeneous ensemble, its soft-voting weights, and bookkeeping fields recording when the checkpoint was created and last accessed. Checkpoints are immutable, so a reuse retrieves the stored ensemble by reference rather than copying it, and only the small weight vector is adjusted when calibration applies. Retrieval cost is therefore independent of model size, which is what makes reuse cheap relative to retraining.

D. Regime Matching

Matching is by regime identifier: on a transition to regime r , the controller tests whether $r \in \mathcal{P}$. In the enhanced variant a fingerprint-similarity gate is additionally applied, so reuse also requires the current window's feature summary to be close to the fingerprint recorded when the checkpoint was created. The gate converts the binary question "has this regime been seen?" into the weaker question "has a sufficiently similar instance of this regime been seen?", which allows the mechanism to decline reuse when a regime recurs under visibly different conditions.

E. Policy Transfer

On a reuse decision the retrieved ensemble becomes the active policy. Its soft-voting weights are optionally recalibrated on the most recent buffered window by measuring the recent accuracy of the random-forest and extra-trees components and renormalizing their weights. Calibration is bounded to the last 50 buffered observations and skipped when fewer than 20 are available, so its cost is fixed and small. When no checkpoint exists, a new policy is trained from a class-anchored buffer and stored. The buffer always retains a stratified slice of the initial training prefix, applied identically to every adaptive model in our comparison, to prevent a refit from collapsing onto a single class when the recent window is homogeneous.

F. Adaptation Trigger

Two monitoring rules are available, and their difference is one of the findings of this work. The *absolute* rule fires when the rolling accuracy of the active policy falls below a fixed floor. The *relative* rule fires when rolling accuracy has dropped by more than a fixed margin relative to a decayed reference built from that same policy's earlier accuracy. The relative rule targets deterioration rather than poor performance, and Section VI-E shows that this distinction has a sharp consequence: a policy that was never good cannot be detected by a relative rule, because the reference decays toward the poor level and the gap the rule looks for never opens.

G. Refresh Mechanism

A refresh rebuilds the active policy from the recent buffer and replaces the corresponding checkpoint, so later reuses of that regime retrieve the refreshed policy. The default refresh is a full refit of the 100-tree ensemble on a 1000-observation buffer. The cost-aware refresh retains the same trigger but refits a smaller 20-tree ensemble on a 300-observation buffer. Section VI-E reports the measurement that motivated this substitution.

H. RAPT-Enhanced

RAPT-Enhanced adds two mechanisms. *Buffer-blended novelty refitting* refits a novel regime on a larger recent buffer of 1500 observations rather than 500, and *selective parity refitting* refits a reused policy from the recent buffer and replaces its checkpoint when rolling accuracy falls below a parity threshold over a short window. Parity refitting addresses the stale-reuse failure directly, because it does not require the regime identifier to change before a retrieved policy can be corrected.

I. Fallback

When no checkpoint exists and the similarity gate declines an available one, the controller trains a fresh policy from the class-anchored buffer and stores it under the incoming regime identifier. The repository grows by one entry per genuinely new regime, and the fallback path is exactly the full-retraining behaviour that RAPT avoids only when it is unnecessary.

V. EXPERIMENTAL METHODOLOGY

A. Datasets

Four network streams are used, summarised in Table I.

5G Campus Network QoS. A processed per-window QoS stream from a 5G campus testbed, with 19 numerical features per window and a three-class target (GOOD, DEGRADED, BAD) from the next window's 90th-percentile delay. Regimes are network scenario labels, and three regimes recur 14 times.

UGR'16. An ISP netflow archive of per-minute traffic aggregates with 134 features, widely used for network intrusion detection evaluation [27]. Its calibration split contains only blacklisted traffic and no labelled attacks, so the labelled 30-day block is used. The target is binary (whether any non-blacklisted attack occurs in the window), and regimes are

METHODOLOGY DIAGRAM PLACEHOLDER

RAPT architecture / stream processing / regime identification / policy repository / similarity / policy reuse / refresh / fallback

Fig. 1. Overview of the proposed Regime-Aware Policy Transfer framework. Incoming stream $\rightarrow$ window construction $\rightarrow$ regime characterization $\rightarrow$ regime identification and similarity $\rightarrow$ policy repository $\rightarrow$ policy reuse $\rightarrow$ performance monitoring $\rightarrow$ refresh trigger $\rightarrow$ cheap or full adaptation $\rightarrow$ updated policy repository, with a fallback path to fresh training when no suitable prior policy exists.

TABLE I
DATASETS USED IN THE EVALUATION. RECURRENCES COUNT REGIME RE-VISITS.

Property	Campus	UGR'16	Nordic	5G NR
Samples	1,799	43,200	91,455	249,500
Features	19	134	16	12
Classes	3	2	3	3
Windows	179	180	182	499
Window size	10	240	500	500
Regimes	3	12	3	4
Recurrences	14	168	17	10

workday/weekend crossed with four-hour time-of-day blocks, giving twelve regimes and 168 recurrence events, the strongest recurrence structure in our evaluation.

NordicDat. Cross-border LTE and 5G predictive-QoS traces released under CC-BY-4.0 [28], with a three-class next-window median-delay target (thresholds fixed on the training prefix) and mobile-network-operator regimes. This is the most weakly predictable stream and the one with the fewest transitions.

5G NR End-to-End Latency. A simulated 5G NR end-to-end latency dataset [29], pre-windowed into 499 windows of 500 packets each and spanning four regimes in the sequence *ABCDACBDAB*, with the same three-class QoS target as the campus stream. It is used for the natural-drift baseline and for the controlled covariate-, concept- and recurring-concept-drift experiments.

B. Preprocessing and Stream Construction

All streams follow the same lightweight pipeline: load, apply the documented target definition, order by timestamp, drop identifier and constant columns, keep numeric features, map infinities to missing, drop rows with missing targets, impute with training-prefix medians, encode categorical targets deterministically, and window. No principal-component analysis, feature selection, hyper-parameter search or global normalization is applied. Target thresholds and scaling statistics are computed on the initial training prefix only.

C. Chronological Ordering and Leakage Control

Every stream is processed in strict chronological order and never shuffled. The initial training prefix is 20% of the windows: 36 windows for the three cross-dataset streams and 99 for the 5G NR latency stream. The prefix is used for initial fitting and is never evaluated as test data. Windows are

predicted before their labels are used for any adaptation, giving the prequential order **predict** $\rightarrow$ **evaluate** $\rightarrow$ **adapt**. Regime annotations are read from the stream and are not derived from the target.

D. Models and Detectors

The five primary models share one base architecture and one retraining buffer, so that differences between them reflect only the adaptation mechanism. The base model is a heterogeneous soft-voting ensemble of a random forest and an extra trees classifier, each with 50 trees of maximum depth 7. *Frozen* trains once on the prefix and never adapts; *Event-Driven* retrains when window accuracy drops below a fixed threshold; *Full Retraining* rebuilds at every regime transition; *RAPT* and *RAPT-Enhanced* are the controllers of Section IV.

ADWIN, Page-Hinkley, EDD and EDMA are evaluated separately as adaptation triggers driving the same retraining harness, with identical model, buffer, capacity and minimum retrain interval; only the trigger differs. Parameters are fixed a priori and not tuned on the test stream (ADWIN $\delta = 0.002$; Page-Hinkley with minimum 30 instances and threshold 50; EDD with warning level 2 and drift level 3; EDMA with $\alpha = 0.2$ and $k = 2.0$). Because ground-truth drift boundaries are not available for all streams, we report detected events, adaptation cost and predictive performance rather than detector precision and recall.

E. Metrics and Statistics

Predictive performance is reported as macro-F1, accuracy, precision and recall. Adaptation cost is reported as *adaptation CPU*, the process time spent inside adaptation routines only, and *runtime*, the total time of the streaming evaluation including feature construction and prediction. Keeping these separate matters because a reduction in adaptation time does not translate proportionally into a reduction in total runtime. We also record retraining events, reuse events, trees trained and trees reused. All experiments use seeds [42, 43, 44, 45, 46] and report mean $\pm$ standard deviation over seeds. Paired comparisons use the Wilcoxon signed-rank test with Cohen's *d*. Where pairing is by window the number of paired observations is 143–146; where it is by seed, $n = 5$ and the smallest attainable two-sided *p*-value is 0.0625. This ceiling is stated wherever it binds, and a non-significant result is reported as a failure to establish a difference rather than as evidence of equivalence. All measurements were taken in a single process

on one machine with $n_jobs=1$, so timings are comparable within the study but are not portable absolute costs.

VI. RESULTS

A. Predictive Performance

Table II reports the primary comparison and Fig. 2 shows the macro-F1 values side by side.

On the 5G Campus stream all five models are close to saturation. Full Retraining recorded the highest macro-F1 at 0.9836 ± 0.0013 , ahead of Event-Driven at 0.9636 ± 0.0006 . RAPT and RAPT-Enhanced both recorded 0.9381 ± 0.0016 , 0.0455 below Full Retraining and significant in that direction ($p = 2.6 \times 10^{-4}$, $d = -0.32$); the two variants agree to four decimal places, so the enhancement produced no measurable change here.

UGR'16 is where the mechanism's assumptions break down, and it is the most informative result in the table. Frozen recorded the highest macro-F1 at 0.9690 ± 0.0029 , consistent with attacks being sparse and temporally clustered. RAPT recorded 0.8360 ± 0.0262 , 0.1236 below Full Retraining ($p = 2.6 \times 10^{-11}$, $d = -0.67$) and the lowest of the five models, despite 133 reuses. The twelve regimes recur frequently and their identifiers remain valid across the 30-day block, but the relationship between netflow features and attack labels does not, so reuse retrieves policies that match the regime label while no longer matching its label semantics. RAPT-Enhanced, which parity-refits a degraded reused policy, recovered to 0.9276 ± 0.0123 , a gain of 0.0917 ($p = 5.4 \times 10^{-9}$, $d = 0.56$) and the only statistically supported improvement of one variant over another in this study.

NordicDat is the weakly predictable stream: Frozen recorded 0.2779 ± 0.0224 while adaptation helps, with Full Retraining at 0.4227 ± 0.0299 and RAPT at 0.3818 ± 0.0184 . RAPT's deficit against Full Retraining is 0.0409 and not significant ($p = 0.255$, $d = -0.12$), though RAPT recorded the highest accuracy here, 0.6172 ± 0.0127 against 0.5694 ± 0.0075 , reflecting a precision-favouring decision profile rather than better macro-F1. On the 5G NR latency stream the five models span only 0.0133 macro-F1, with Full Retraining leading at 0.9027 ± 0.0059 and RAPT lowest at 0.8894 ± 0.0110 ; the difference is significant on accuracy ($p = 0.0043$) but small in magnitude.

The predictive results do not support a claim that RAPT improves accuracy: on three of four streams Full Retraining records the highest macro-F1, and RAPT is below it on all four. The case for the mechanism rests on cost and on the conditions under which reuse is safe.

B. Computational Cost and Policy Reuse

Table II also reports adaptation and total cost, and Fig. 3 shows the adaptation-CPU comparison. The reduction relative to Full Retraining is consistent in direction across all four streams: 1.4008 to 0.2292 s on the 5G Campus stream (-83.6%), 23.6318 to 1.9711 s on UGR'16 (-91.7%), 2.2034 to 0.2594 s on NordicDat (-88.2%), and 0.7636 to 0.3955 s on the 5G NR stream (-48.2%). Reductions in total runtime are

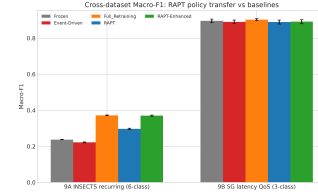


Fig. 2. Macro-F1 for the five primary models on the two largest streams of the cross-dataset comparison. RAPT tracks the frozen baseline on UGR'16 rather than the retrained models, the signature of stale policy reuse; on the 5G NR latency stream the five models are within 0.0133 macro-F1.

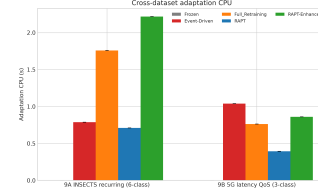


Fig. 3. Adaptation CPU on the two largest streams of the cross-dataset comparison. The ordering is inverted relative to Fig. 2: RAPT uses roughly an order of magnitude less adaptation time than Full Retraining on UGR'16 while recording substantially lower macro-F1.

smaller in every case (-44.0% , -82.9% , -40.6% , -14.5%), the gap being the prediction and feature-construction cost that adaptation does not touch. That gap is largest on the 5G NR stream, where Full Retraining performs only nine retrains.

The event counts explain where the saving comes from. Frozen performs no adaptation and Event-Driven performs one to eleven retrains depending on the stream, while Full Retraining retrains at every regime transition (14, 144, 18 and 9). RAPT performs 2, 11, 2 and 3 retrains while retrieving a stored policy 12, 133, 16 and 6 times, and UGR'16 shows the most extreme ratio at 133 reuses against 11 retrains.

C. Cross-Dataset Behaviour

Comparing streams isolates which findings depend on the data. The adaptation-cost reduction and the predominance of reuse over retraining appear on all four, so both are properties of the mechanism rather than of one stream. The predictive outcome is not: RAPT-Enhanced improves over base RAPT by 0.0917 on UGR'16 and by 0.0021 on the 5G NR stream, and the two are identical on the 5G Campus stream. The enhancement matters where regimes recur and their label relationship changes, and is inert otherwise: UGR'16 has 168 recurrence events across twelve regimes while the 5G Campus stream has 14 across three.

Reuse frequency alone says nothing about whether a retrieved policy was appropriate. UGR'16 is the counter-example, where RAPT reuses 133 times and records the lowest macro-F1 of the five models while RAPT-Enhanced reuses the same 133 times and records 0.9276. The two differ not in how often they reuse but in what happens afterwards, since the enhanced variant re-examines a reused policy and refits it when recent accuracy falls below the parity threshold. The same reuse frequency therefore yields a 0.0917 difference,

TABLE II

PRIMARY MODEL COMPARISON AND COMPUTATIONAL COST, MEAN OVER SEEDS 42–46 UNDER THE PREQUENTIAL PROTOCOL. BOLD MARKS THE BEST MACRO-F1 PER STREAM. ADAPT. CPU AND RUNTIME ARE IN SECONDS; RETRAINING AND REUSE EVENTS ARE COUNTS.

Dataset	Model	F1	Acc.	Prec.	Rec.	CPU (s)	Run (s)	Retr.	Reuse
5G Campus	Frozen	0.9364 $\pm$ 0.0023	0.9822 $\pm$ 0.0006	0.9379 $\pm$ 0.0025	0.9377 $\pm$ 0.0017	0.0000	1.2482	0	–
	Event-Driven	0.9636 $\pm$ 0.0006	0.9908 $\pm$ 0.0003	0.9637 $\pm$ 0.0008	0.9646 $\pm$ 0.0002	0.1080	1.3926	1	–
	Full Retraining	0.9836 $\pm$ 0.0013	0.9951 $\pm$ 0.0000	0.9838 $\pm$ 0.0015	0.9843 $\pm$ 0.0012	1.4008	2.6677	14	–
	RAPT	0.9381 $\pm$ 0.0016	0.9838 $\pm$ 0.0003	0.9393 $\pm$ 0.0017	0.9392 $\pm$ 0.0016	0.2292	1.4931	2	12
	RAPT-Enhanced	0.9381 $\pm$ 0.0016	0.9838 $\pm$ 0.0003	0.9393 $\pm$ 0.0017	0.9392 $\pm$ 0.0016	0.2337	2.1046	2	12
UGR'16	Frozen	0.9690 $\pm$ 0.0029	0.9899 $\pm$ 0.0007	0.9795 $\pm$ 0.0030	0.9662 $\pm$ 0.0028	0.0000	2.4914	0	–
	Event-Driven	0.8972 $\pm$ 0.0277	0.9898 $\pm$ 0.0013	0.9042 $\pm$ 0.0269	0.8955 $\pm$ 0.0280	0.5754	3.1303	3	–
	Full Retraining	0.9595 $\pm$ 0.0012	0.9873 $\pm$ 0.0007	0.9726 $\pm$ 0.0014	0.9580 $\pm$ 0.0015	23.6318	26.2627	144	–
	RAPT	0.8360 $\pm$ 0.0262	0.9635 $\pm$ 0.0182	0.8527 $\pm$ 0.0205	0.8336 $\pm$ 0.0294	1.9711	4.4990	11	133
	RAPT-Enhanced	0.9276 $\pm$ 0.0123	0.9880 $\pm$ 0.0017	0.9375 $\pm$ 0.0117	0.9251 $\pm$ 0.0123	2.7182	5.9329	11	133
NordicDat	Frozen	0.2779 $\pm$ 0.0224	0.4770 $\pm$ 0.0123	0.4215 $\pm$ 0.0226	0.2487 $\pm$ 0.0229	0.0000	2.5488	0	–
	Event-Driven	0.2547 $\pm$ 0.0177	0.4583 $\pm$ 0.0367	0.3909 $\pm$ 0.0208	0.2274 $\pm$ 0.0151	0.2533	2.8435	2	–
	Full Retraining	0.4227 $\pm$ 0.0299	0.5694 $\pm$ 0.0075	0.4938 $\pm$ 0.0233	0.4085 $\pm$ 0.0293	2.2034	4.7593	18	–
	RAPT	0.3818 $\pm$ 0.0184	0.6172 $\pm$ 0.0127	0.4703 $\pm$ 0.0185	0.3582 $\pm$ 0.0192	0.2594	2.8257	2	16
	RAPT-Enhanced	0.3783 $\pm$ 0.0321	0.5677 $\pm$ 0.0222	0.4600 $\pm$ 0.0307	0.3614 $\pm$ 0.0318	2.6492	5.8812	2	16
5G NR Lat.	Frozen	0.8961 $\pm$ 0.0099	0.9075 $\pm$ 0.0061	0.9314 $\pm$ 0.0083	0.8798 $\pm$ 0.0085	0.0000	1.7862	0	–
	Event-Driven	0.8903 $\pm$ 0.0107	0.9055 $\pm$ 0.0060	0.9475 $\pm$ 0.0085	0.8688 $\pm$ 0.0083	1.0406	2.7561	11	–
	Full Retraining	0.9027 $\pm$ 0.0059	0.9145 $\pm$ 0.0045	0.9478 $\pm$ 0.0053	0.8829 $\pm$ 0.0055	0.7636	2.4689	9	–
	RAPT	0.8894 $\pm$ 0.0110	0.9025 $\pm$ 0.0068	0.9258 $\pm$ 0.0103	0.8737 $\pm$ 0.0092	0.3955	2.1112	3	6
	RAPT-Enhanced	0.8915 $\pm$ 0.0117	0.9055 $\pm$ 0.0076	0.9348 $\pm$ 0.0106	0.8737 $\pm$ 0.0098	0.8609	2.5710	3	6

locating the failure in the reuse decision rather than in reuse as such. On the 5G Campus stream the two are identical, so the parity refit never fired and the 0.0455 gap to Full Retraining persists in both.

The NordicDat results resist a single reading: all models perform poorly in absolute terms and the ordering of accuracy and macro-F1 inverts, with RAPT recording the highest accuracy at 0.6172 while Full Retraining records the highest macro-F1 at 0.4227. Whether this inversion is a property of streams with weak baselines or of this target construction cannot be separated with the present data, and we do not merge the two into one claim.

D. Efficiency Ablation

Table III reports the mechanism ladder on the 5G Campus stream, and Fig. 4 shows the accuracy–cost position of each configuration. Each row adds one mechanism to the preceding row.

RAPT-Full, the base controller with the similarity gate, recorded 0.9587 ± 0.0000 macro-F1 at 0.4283 s adaptation CPU. Replacing the absolute refit trigger with a relative-degradation trigger (RAPT-Evidence) produced an identical macro-F1 and fired *zero* refreshes, while replacing it with an absolute accuracy floor (RAPT-Floor) fired 7.6 refreshes on average and recorded 0.9797 ± 0.0050 at 0.5865 s, above RAPT-Full but with higher seed variance.

RAPT-Cheap retains the periodic refresh schedule but substitutes the cheaper refit, recording 0.9851 ± 0.0029 macro-F1 at 0.8477 s adaptation CPU against Full Retraining’s 0.9829 ± 0.0023 at 1.4063 s. The macro-F1 difference is $\Delta = +0.0022$ with $p = 0.3125$, $d = 0.47$ and a 95% confidence interval of $[-0.0036, +0.0080]$. The interval contains zero, so the comparison does not establish a difference. Relative to the full-refit periodic refresh, which recorded 0.9847 ± 0.0028

TABLE III

RAPT EFFICIENCY ABLATION ON THE 5G CAMPUS STREAM, MEAN OVER SEEDS 42–46. FULL RETRAINING IS A REFERENCE ROW. REFRESHES COUNTS BOUNDED IN-PLACE POLICY REBUILDS; RAPT-INCREMENTAL INSTEAD UPDATES AN ONLINE LEARNER.

Configuration	Macro-F1	Adapt. CPU	Refreshes
Full Retrain. (ref.)	0.9829 $\pm$ 0.0023	1.4063	–
RAPT-Full	0.9587 $\pm$ 0.0000	0.4283	0
RAPT-Evidence	0.9587 $\pm$ 0.0000	0.4260	0
RAPT-Floor	0.9797 $\pm$ 0.0050	0.5865	7.6
RAPT-Cheap	0.9851 $\pm$ 0.0029	0.8477	22
RAPT-Incremental	0.9549 $\pm$ 0.0020	0.5813	0

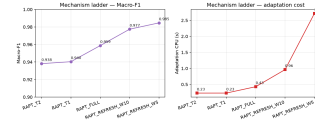


Fig. 4. The RAPT mechanism ladder. RAPT-Cheap reaches the accuracy of Full Retraining at roughly 40% lower adaptation cost, while RAPT-Incremental sits below RAPT-Full on accuracy at higher cost.

at 2.7168 s, the cheap refit retains accuracy while cutting adaptation CPU by roughly 69%.

RAPT-Incremental replaced the batch refresh with an online learner blended into the reused policy and recorded 0.9549 ± 0.0020 at 0.5813 s, below RAPT-Full on accuracy and above it on cost, so it is dominated on both axes by a configuration that does not use incremental learning.

The supported statement about the cost-aware configuration is that it *matched* Full Retraining’s predictive performance at substantially lower adaptation cost. The macro-F1 difference is numerically positive but not statistically significant, so no claim of superiority is made; the evidence for RAPT-Cheap is the cost reduction, not the accuracy difference.

E. Cost Drivers and Trigger Behaviour

The cheap refresh was chosen from a direct measurement rather than an assumption about which factor dominates refit cost. Holding the ensemble fixed and varying one factor at a time, a 20-tree refit cost approximately 22 ms and a 100-tree refit approximately 104 ms, whereas a 200-observation buffer cost approximately 80 ms and a 1000-observation buffer approximately 104 ms. Tree count therefore dominated buffer size here, and reducing it from 100 to 20 removed roughly four fifths of the refit cost, which is what makes a periodic refresh affordable. This is a property of the evaluated configuration, not a general law: an ensemble whose per-tree or buffer-processing cost differs would shift the balance.

The zero-refresh result for RAPT-Evidence (Section VI-D) is a mechanism-level finding rather than a tuning outcome. A relative trigger asks whether current accuracy has fallen by more than a margin against a decayed reference built from that policy’s own earlier accuracy. When a regime is persistently poor rather than deteriorating, the reference decays toward the poor level and the gap the rule tests for never opens, so the trigger stays silent even though the policy is unfit. RAPT-Floor detected the same condition against a fixed floor and fired 7.6 refreshes where the relative rule fired none. The floor is stream-specific: the value used here was selected on the evaluated stream and would require calibration elsewhere, which is why it is presented as a mechanism to be tuned rather than a recommended constant.

F. Historical Detector Comparison

Table IV compares RAPT against the four detectors, and Fig. 5 shows the macro-F1 comparison.

The most immediate result is that three of the four detectors did not fire: ADWIN, Page-Hinkley and EDMA recorded zero adaptation events on all three streams and therefore reproduced the frozen baseline exactly (0.9364 on the 5G Campus stream, 0.9690 on UGR’16, 0.2779 on NordicDat). With the error signal they were given and parameters fixed a priori, they were not sensitive enough to trigger adaptation here, so their performance is by construction that of a model that never adapts.

EDD fired 38 or 39 times per stream and recorded the highest macro-F1 of any method on the 5G Campus stream (0.9847 ± 0.0019) and on NordicDat (0.4184 ± 0.0165); on UGR’16 it recorded 0.9655 ± 0.0016 , marginally below the untriggered baseline. Its adaptation cost was the highest in the comparison, at 3.98, 6.10 and 4.69 s.

RAPT occupied a different point in the trade-off, recording 0.9381, 0.8360 and 0.3818 with 2, 11 and 2 adaptation events and adaptation CPU of 0.2292, 1.9711 and 0.2594 s. Against EDD it was cheaper on all three streams and more accurate on none in macro-F1 terms, though on the 5G Campus stream it exceeded the three non-firing detectors by 0.0017. EDD buys accuracy with adaptation events, the conservative detectors buy nothing by never adapting, and RAPT sits between them at the lowest adaptation cost among the methods that adapt at all.

TABLE IV

HISTORICAL DRIFT DETECTORS AS ADAPTATION TRIGGERS, MEAN $\pm$ SD OVER SEEDS 42–46. DETECTORS DRIVE THE SAME RETRAINING HARNESS AS THE PRIMARY MODELS. EVENTS COUNTS DETECTIONS THAT LED TO ADAPTATION; ADAPT. CPU IS IN SECONDS.

Dataset	Method	F1	CPU (s)	Events
5G Campus	ADWIN	0.9364 ± 0.0023	0.0000	0
	Page-Hinkley	0.9364 ± 0.0023	0.0000	0
	EDMA	0.9364 ± 0.0023	0.0000	0
	EDD	0.9847 ± 0.0019	3.9824	38
	RAPT	0.9381 ± 0.0016	0.2292	2
UGR’16	ADWIN	0.9690 ± 0.0029	0.0000	0
	Page-Hinkley	0.9690 ± 0.0029	0.0000	0
	EDMA	0.9690 ± 0.0029	0.0000	0
	EDD	0.9655 ± 0.0016	6.1012	39
	RAPT	0.8360 ± 0.0262	1.9711	11
NordicDat	ADWIN	0.2779 ± 0.0224	0.0000	0
	Page-Hinkley	0.2779 ± 0.0224	0.0000	0
	EDMA	0.2779 ± 0.0224	0.0000	0
	EDD	0.4184 ± 0.0165	4.6875	39
	RAPT	0.3818 ± 0.0184	0.2594	2

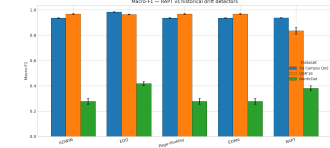


Fig. 5. Macro-F1 against the historical drift detectors. ADWIN, Page-Hinkley and EDMA recorded no adaptation events and coincide with the frozen baseline.

G. Statistical Analysis

The paired comparisons support a narrow set of conclusions. On UGR’16, RAPT’s deficit against Full Retraining is significant ($p = 2.6 \times 10^{-11}$, $d = -0.67$) and RAPT-Enhanced’s recovery over RAPT is significant ($p = 5.4 \times 10^{-9}$, $d = 0.56$), both with window-level pairing over 144 observations. On the 5G Campus stream, RAPT’s deficits against Full Retraining ($p = 2.6 \times 10^{-4}$, $d = -0.32$) and Event-Driven ($p = 5.7 \times 10^{-3}$) are significant, and on NordicDat its deficit against Full Retraining is not ($p = 0.255$) while its advantage over Event-Driven is ($p = 1.8 \times 10^{-4}$); on the 5G NR stream the differences against Full Retraining and Frozen are significant but small.

One caveat applies throughout: the window-level tests treat windows from one stream as independent observations, which overstates the effective sample size for claims intended to generalise beyond the evaluated streams, and the five-seed ablation comparisons against Full Retraining ($p = 0.3125$ and $p = 0.375$) are limited by power as well as by effect size, so we report them as non-significant.

VII. DISCUSSION

The cost result has a straightforward explanation. Retraining at a regime transition rebuilds the ensemble from a buffer, and that cost scales with tree count and buffer size. Reuse retrieves an ensemble built earlier and stored by reference, so its cost is a dictionary lookup plus a bounded weight calibration. Where

regimes recur and their label relationships hold, the retrieved policy is adequate and the rebuild was unnecessary, so the saving tracks how often regimes recur.

RAPT-Cheap worked for a measurable reason. The refit-cost probe showed tree count dominating buffer size for this ensemble, so a refresh rebuilding 20 trees on a 300-observation buffer retains enough capacity to correct a policy already close to the target while costing a fraction of the full refit. The periodic schedule is what makes it effective on this stream, since a reused policy would otherwise remain frozen for the whole duration of a recurring regime; neither mechanism alone accounts for the result.

The incremental mechanism failed, and the diagnostics locate the failure. A standalone incremental ensemble reached approximately 0.945 macro-F1 against approximately 0.98 for the batch ensemble, and adding trees did not close the gap (one tree 0.946, three 0.945, ten 0.941) while cost grew. Blending the incremental learner into the reused batch policy was also unhelpful, reducing macro-F1 to approximately 0.963 at weight 0.5 and leaving it at approximately 0.983 at weight 0.3, effectively the batch-only result. The batch ensemble is already close to ceiling on a 19-feature, three-class problem, leaving little headroom for a second learner. This is a finding about this stream and problem size, not a general claim about incremental learning.

The trigger finding generalises more cleanly than the incremental one, because it follows from the definition of the rule rather than from the data. A relative trigger compares a policy against its own past, so a policy that was always poor has no relative drop to detect. This is a structural blind spot rather than a tuning problem, and it is why the absolute floor was introduced, though that floor needs per-stream calibration, whereas the relative rule needs no constant but cannot see the case that matters most for a reused policy.

The cross-dataset results point to recurrence as the condition under which the mechanism is worth deploying. The cost saving appears wherever reuse occurs, but the accuracy penalty is concentrated on UGR'16, where regimes recur while their label relationship does not. That is precisely where a reused policy is stale and where the parity refit earns its place, converting 0.8360 into 0.9276, so policy reuse should be paired with a check on the reused policy's own recent performance; reuse on the strength of the regime identifier alone is unsafe when label semantics can drift independently of it.

The detector comparison shows that the choice between adaptation strategies is a choice of operating point rather than of winner. The conservative detectors never fired on these streams and matched the frozen baseline, which is defensible when the stream is stationary and a failure when it is not, while EDD tracked the drift at the highest adaptation cost. RAPT sits at the low-cost end among the methods that adapt, and what distinguishes it is a different operating point: it spends adaptation budget on retrieval rather than on rebuilding. The conditions for RAPT to be useful follow with some precision. The stream must provide a usable regime signal, regimes must recur often enough for the repository to be hit, and the

label relationship within a regime must be stable or monitored so a stale retrieval can be corrected. Without recurrence the mechanism reduces to retraining with additional bookkeeping, and where label semantics drift independently of the regime label, reuse without monitoring is worse than not reusing at all, as UGR'16 shows.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The evaluation uses five seeds, capping the attainable two-sided p -value at 0.0625 for seed-level comparisons, so several comparisons with large effect sizes remain formally non-significant and the ablation results in particular are limited by power rather than by effect size alone. Window-level tests have more observations but treat windows from a single stream as independent, overstating the effective sample size for claims intended to generalise. Three of the four streams are used as released, but the drift-severity study on the 5G NR latency stream applies controlled transformations at known points, so its severity response is measured on constructed drift. Timings come from one environment with parallelism disabled and are internally comparable but not portable absolute costs.

Further limitations follow from the design. The mechanism depends on regime recurrence, which is measured rather than assumed; a stream without recurrence would never exercise the repository, and a genuinely novel regime falls back to retraining with no advantage over the baseline. The absolute floor used by RAPT-Floor is a fixed threshold calibrated on the evaluated stream and the 20-tree cheap refit is a configuration chosen for this ensemble, so both would need re-examination where the pre- and post-drift difference is larger and a reduced-capacity refit could underfit. A policy retrieved for a regime whose label relationship has changed is stale, and only the parity refit in the enhanced variant addresses this. The repository grows with the number of distinct regimes and is never pruned, and the approach assumes regime similarity is meaningful, which holds for semantically defined regimes but would be questionable if the regime signal were noisy. Class imbalance remains a factor throughout, and on UGR'16 the sparse target is part of why the frozen baseline is strong there; the class-anchored buffer prevents degenerate single-class refits but does not remove the imbalance.

IX. CONCLUSION

Conventional drift adaptation pays for adaptation by retraining, and where operating conditions recur that expenditure is partly redundant. RAPT retains the adaptation policy learned for each regime and retrieves it when the regime returns, falling back to retraining only when no suitable policy exists and refreshing a retrieved policy when monitoring shows it has stopped fitting. Across four network streams the mechanism reduced adaptation CPU relative to full retraining by 48% to 92%, with the largest savings where regimes recur most often. The saving did not come with an accuracy improvement: RAPT recorded lower macro-F1 than full retraining on all four streams, and where regimes recur but their label relationship changes, blind reuse was actively harmful, recovering only

when a parity-refit mechanism was added. A cost-aware configuration matched full retraining within measurement noise ($\Delta_{\text{macro-F1}} = +0.0022$, $p = 0.31$) at roughly 40% lower adaptation cost, the strongest cost-efficiency result in the study.

Two mechanisms were rejected on measured evidence: a relative-degradation trigger cannot detect a policy that was always poor, because its reference decays to the poor level, so an absolute floor is needed instead, and an incremental learner was dominated on both accuracy and cost by a batch configuration without incremental updating, which the diagnostics attribute to the batch ensemble already being near ceiling here. Policy reuse is therefore worth deploying where regimes recur and label semantics are stable or monitored, and the natural extension is to make the reuse decision adaptive: a controller that estimates whether a retrieved policy's regime is still semantically the same regime, rather than whether its identifier matches, would address the failure mode observed on UGR'16 directly. Whether that estimate can be made without labels is left open.

REFERENCES

- [1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
- [2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2018.
- [3] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, and K. Ghédira, "Discussion and review on evolving data streams and concept drift adapting," *Evolving Systems*, vol. 9, no. 1, pp. 1–23, 2018.
- [4] J. C. Schlimmer and R. H. Granger, Jr., "Incremental learning from noisy data," *Machine Learning*, vol. 1, no. 3, pp. 317–354, 1986.
- [5] G. Widmer, "Combining robustness and flexibility in learning drifting concepts," in *Proc. European Conf. Artificial Intelligence (ECAI)*, 1994, pp. 468–472.
- [6] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, and T. Abdesslem, "Adaptive random forests for evolving data stream classification," *Machine Learning*, vol. 106, no. 9–10, pp. 1469–1495, 2017.
- [7] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448.
- [8] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [9] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Proc. Brazilian Symp. Artificial Intelligence (SBIA)*, 2004, pp. 286–295.
- [10] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Proc. Int. Workshop on Knowledge Discovery from Data Streams*, 2006, pp. 77–86.
- [11] R. S. M. Barros, D. R. L. Cabral, P. M. Gonçalves, Jr., and S. G. T. C. Santos, "RDDM: Reactive drift detection method," *Expert Systems with Applications*, vol. 90, pp. 344–355, 2017.
- [12] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognition Letters*, vol. 33, no. 2, pp. 191–198, 2012.
- [13] I. Frias-Blanco, J. del Campo-Ávila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Diaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.
- [14] D. M. dos Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov-Smirnov test," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1545–1554.
- [15] T. Brockhoff, M. S. Uysal, and W. M. P. van der Aalst, "Time-aware concept drift detection using the earth mover's distance," in *Proc. Int. Conf. Process Mining (ICPM)*, 2020, pp. 33–40.
- [16] L. Yang and A. Shami, "A lightweight concept drift detection and adaptation framework for IoT data streams," *IEEE Internet of Things Magazine*, vol. 4, no. 2, pp. 96–101, 2021.
- [17] J. Z. Kolter and M. A. Maloof, "Dynamic weighted majority: An ensemble method for drifting concepts," *Journal of Machine Learning Research*, vol. 8, no. 91, pp. 2755–2790, 2007.
- [18] D. P. Helmbold and P. M. Long, "Tracking drifting concepts by minimizing disagreements," *Machine Learning*, vol. 14, no. 1, pp. 27–45, 1994.
- [19] D. Brzeziński and J. Stefanowski, "Reacting to different types of concept drift: The accuracy updated ensemble algorithm," *IEEE Trans. Neural Networks and Learning Systems*, vol. 25, no. 1, pp. 81–94, 2014.
- [20] A. Abbasi, A. R. Javed, C. Chakraborty, J. Nebhen, W. Zehra, and Z. Jalil, "ElStream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning," *IEEE Access*, vol. 9, pp. 66408–66419, 2021.
- [21] L. Yang, D. M. Manias, and A. Shami, "PWPAE: An ensemble framework for concept drift adaptation in IoT data streams," in *Proc. IEEE Global Communications Conf. (GLOBECOM)*, 2021, pp. 1–6.
- [22] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2000, pp. 71–80.
- [23] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdesslem, and A. Bifet, "River: machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
- [24] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [25] S. Markovitch and P. D. Scott, "The role of forgetting in learning," in *Machine Learning Proceedings 1988*. San Francisco, CA, USA: Morgan Kaufmann, 1988, pp. 459–465.
- [26] M. Tambe and A. Newell, "Some chunks are expensive," in *Machine Learning Proceedings 1988*. San Francisco, CA, USA: Morgan Kaufmann, 1988, pp. 451–458.
- [27] G. Maciá-Fernández, J. Camacho, R. Magán-Carrión, P. García-Teodoro, and R. Therón, "UGR'16: A new dataset for the evaluation of cyclostationarity-based network IDSs," *Computers & Security*, vol. 73, pp. 411–424, 2018.
- [28] NordicDat cross-border predictive QoS dataset, Zenodo record 10964584, released under CC-BY-4.0.
- [29] 5G NR end-to-end latency simulation dataset, Zenodo record 20035549.